Fixed-Point vs. Floating-Point in IIR Filtering Does the Format Choice Actually Matter?

1st Sepehr Farrokhi
Hanze University of Applied Sciences
s.farrokhi.koochesfahani@st.hanze.nl

2nd Amin Javan
Hanze University of Applied Sciences
ma.javan@st.hanze.nl

3rd Armin Siavashi
Hanze University of Applied Sciences
a.siavashi@st.hanze.nl

***Abstract*—When designing a DSP system, the choice between fixed-point and floating-point arithmetic directly affects accuracy, memory consumption, and hardware cost. This assignment evaluates both dimensions using a 4th-order Butterworth IIR low-pass filter as a testbed. Two fixed-point formats (Q3.13 and Q8.8) and two floating-point formats (float16 and float32) are simulated and compared against a float64 high-precision reference, not a true ground truth, but a sufficiently accurate baseline for quantifying representational error. Accuracy is measured via MSE and SNR; efficiency is assessed through memory footprint and a qualitative hardware cost analysis. The most striking finding is that format selection *within* fixed-point matters greatly: Q3.13 achieves 44.87 dB SNR while Q8.8 reaches only 13.84 dB, despite identical 16-bit word length and 2000 bytes of memory. With a faithful IEEE 754 half-precision SOS simulation, float16 attains 53.68 dB SNR at the same 2000-byte signal footprint, outperforming Q3.13 yet remaining far below float32. Fixed-point offers lower hardware cost and power consumption on embedded targets; floating-point trades those efficiencies for higher precision and simpler programming where an FPU is available.**

***Index Terms*—fixed-point, floating-point, IIR filter, quantisation error, Q-format, IEEE 754, DSP**

I. Introduction

Every DSP system encodes numbers in finite binary words. Whether those words use fixed-point or floating-point arithmetic determines accuracy, memory consumption, and hardware cost—three objectives that often conflict depending on the application. This decision is made early and is difficult to reverse: a processor either has a floating-point unit or it does not.

Neither paradigm is universally better. Fixed-point dominates in hearing aids, sensor nodes, and motor controllers where power and cost are the binding constraints [1], [2]. Floating-point is preferred wherever precision matters and hardware cost is secondary [3]. What is less obvious is that the choice of format *within* fixed-point, how many bits go to the integer part versus the fractional part, can matter more than the fixed vs. floating-point distinction itself. The choice of binary-point position within a fixed-point word directly determines the trade-off between representable range and resolution [6]. This assignment tests that claim directly.

We evaluate four representations applied to a 4th-order Butterworth IIR low-pass filter: Q3.13 (13 fractional bits in a 16-bit word), Q8.8 (8 fractional bits), float16 (10 mantissa bits), and float32 (23 mantissa bits). An IIR topology was chosen because its recursive feedback re-introduces rounding errors on every output cycle, making it a more sensitive testbed than a non-recursive FIR design [1]. For each format we measure MSE, SNR, and memory footprint against a float64 high-precision baseline.

We address two research questions: (1) What are the main characteristics, advantages, and disadvantages of fixed-point and floating-point representations for IIR filter processing, in terms of accuracy and efficiency? (2) What quantisation errors are observed for Q3.13, Q8.8, float16, and float32?

II. Background

A. Fixed-Point Representation

A Qm.n number stores m integer bits and n fractional bits in a signed $(m+n)$-bit word [1]. Resolution is 2^{-n} uniformly across the range $[-2^{m-1},\, 2^{m-1} - 2^{-n}]$. Because the binary point never moves, the programmer must choose a format whose range covers all signal values before filtering begins.

Q3.13 uses 13 fractional bits in a 16-bit word: range $[-4, 4)$, resolution $\approx 1.2 \times 10^{-4}$. It is the smallest format tested that avoided overflow for this filter's coefficients, whose coefficients span approximately $[-1.32, 2.0]$; Q1.15 was initially considered but rejected because its range of $[-1, 1)$ cannot represent these coefficients without overflow. Q8.8 uses 8 integer and 8 fractional bits: range $[-128, 128)$, resolution $\approx 3.9 \times 10^{-3}$, 32 times coarser than Q3.13. Both use identical 16-bit words and identical memory; only the binary point position differs.

On hardware, integer multiply-accumulate units are smaller and more power-efficient than floating-point units, making fixed-point the standard choice for microcontrollers without an FPU [2]. The precision cost is that coefficient quantisation in IIR filters shifts pole locations, which alters the frequency response. Poles close to the unit circle, typical in selective low-pass designs, are especially sensitive to this shift [6].

B. Floating-Point Representation

IEEE 754 floating-point encodes a value as $(-1)^s \times 1.M \times 2^{E-\text{bias}}$, where s is the sign, M the mantissa, and E the biased exponent [4]. The standard defines formats, rounding modes, and exception behaviour; a widely used tutorial exposition is [5]. The exponent adapts to the signal magnitude automatically, giving roughly constant relative precision across a wide dynamic range. no manual scaling required.

float16 provides 10 mantissa bits (≈3 decimal digits, max 65504). float32 provides 23 mantissa bits (≈7 decimal digits, max $\approx 3.4 \times 10^{38}$). The efficiency cost is hardware area and power: floating-point units require significantly more silicon and draw more power than integer units, making fixed-point the standard choice for microcontrollers without a dedicated FPU [2]. Floating-point precision comes at a real hardware price.

III. Simulation Setup

An IIR filter was chosen because its recursive structure re-introduces rounding errors on every output sample, making quantisation effects more pronounced than in a non-recursive FIR design [1]. A 4th-order Butterworth was selected: complex enough for pole sensitivity to manifest under quantisation, yet simple enough to isolate quantisation as the dominant error source. The maximally flat passband ensures deviations from the float64 reference reflect quantisation error rather than passband ripple.

SOS form was chosen over direct-form II because cascading second-order sections limits rounding error propagation between stages; direct-form II accumulates rounding across all poles, inflating error beyond what the number format alone causes [6]. The four formats form two symmetric pairs — Q3.13/Q8.8 isolate the effect of binary-point position at fixed word length; float16/float32 isolate mantissa width at fixed exponent range.

Fixed-point filtering used spfpm [7]. Floating-point used `scipy.signal.sosfilt` [8] with NumPy `float32` [9]. Since `sosfilt` does not support `float16`, half-precision was implemented as a sample-by-sample direct-form II SOS cascade with each multiply–add explicitly rounded in IEEE 754 binary16 [4], ensuring true half-precision behaviour rather than post-hoc quantisation.

All outputs are compared against a float64 baseline; with 52 mantissa bits its rounding error is negligible relative to the formats under test [4].

IV. Results

TABLE I
Quantisation error metrics for the IIR filter (1000 samples, $f_c = 100$ Hz). Values recomputed with faithful float16 SOS simulation.

Format	MSE	SNR (dB)	Memory
Q3.13 (fixed)	4.04×10^{-6}	44.87	2000 B
Q8.8 (fixed)	5.12×10^{-3}	13.84	2000 B
float16 (float)	5.31×10^{-7}	53.68	2000 B
float32 (float)	2.71×10^{-15}	136.60	4000 B

Fig. 1 shows per-sample quantisation error $e[n] = y_{\text{ref}}[n] - \hat{y}[n]$ for all four formats . Note the different vertical scales: Q8.8 error is on the order of 10^{-1}; float32 error is near 10^{-7}.

Fig. 2 shows the *distribution* of the same error sequence. Fixed-point histograms are wider, reflecting coarse uniform quantisation steps; float16 is intermediate; float32 concentrates tightly around zero.

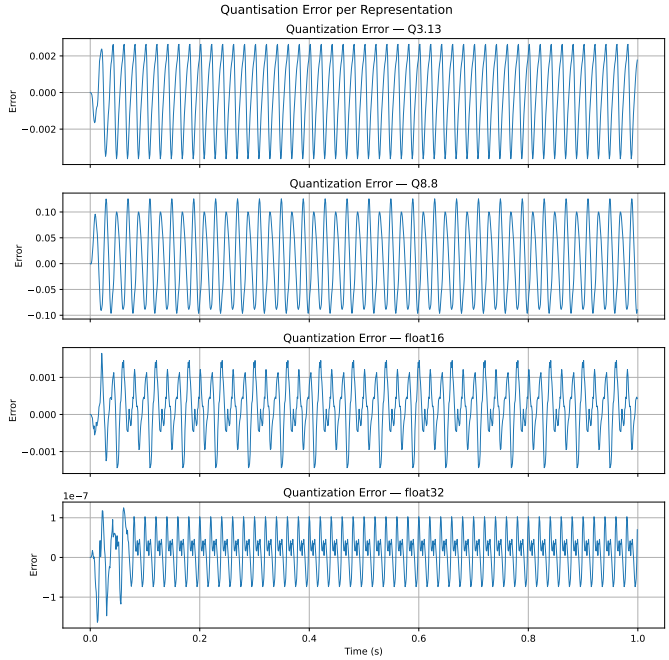

Fig. 1. Per-sample quantisation error for each representation.

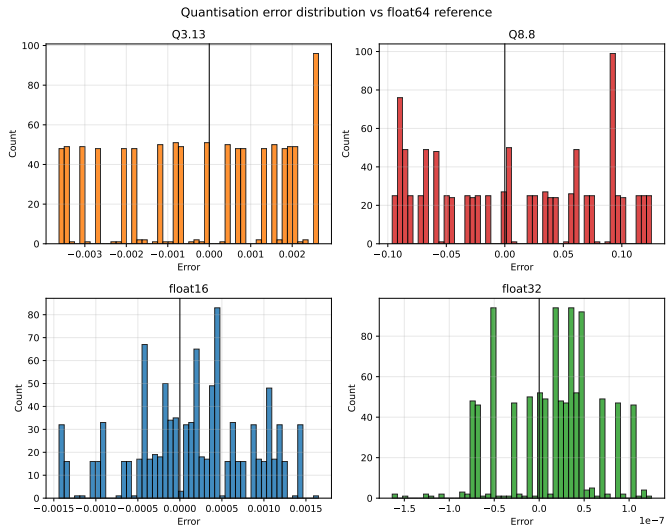

Fig. 2. Histogram of $e[n] = y_{\text{ref}}[n] - \hat{y}[n]$ for each representation.

Fig. 3 shows the frequency-response magnitude for each quantised SOS filter vs. the float64 reference. Q8.8 shows visible deviation in both passband and stopband.

Fig. 4 summarises SNR across formats.

V. Discussion

A. RQ1: Characteristics, Advantages, and Disadvantages

Fixed-point and floating-point differ along two axes: efficiency and accuracy. On efficiency, fixed-point wins for embedded targets: a 16-bit integer multiplier is smaller and more power-efficient than a 32-bit FPU; Analog Devices estimate fixed-point DSP cores cost roughly half as much

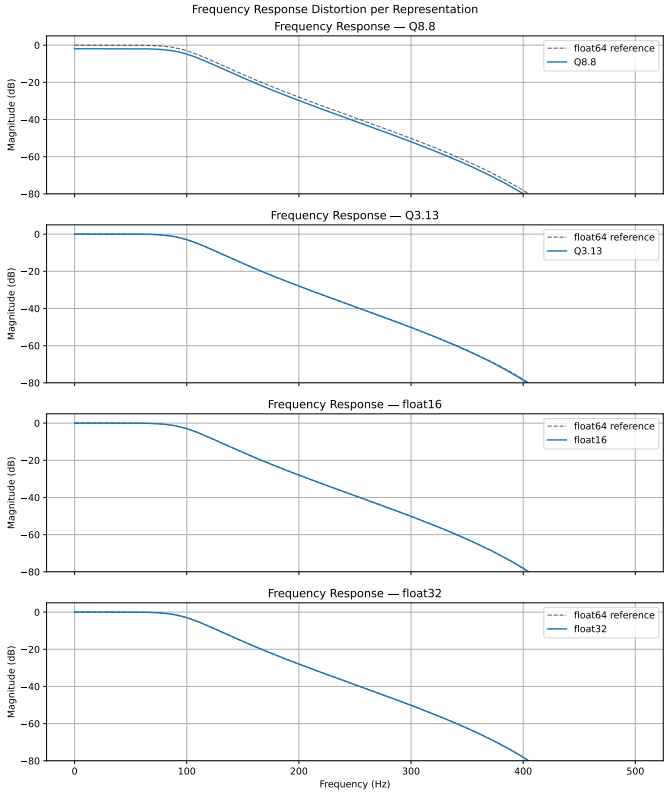

Fig. 3. Magnitude response vs. float64 reference.

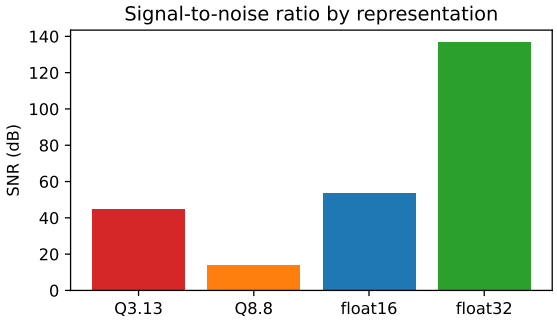

Fig. 4. SNR (dB) across all four representations.

and draw significantly less power [2]. For battery-powered devices that is often the deciding factor. IIR filters are more sensitive to fixed-point quantisation than FIR filters because recursive feedback accumulates rounding error on every sample, whereas FIR errors are bounded by the filter length [1].

On accuracy, floating-point's adaptive exponent means precision scales with signal magnitude automatically, no manual scaling, no hard overflow for moderate signal levels. Fixed-point requires bit allocation to be decided upfront. In IIR filter design this matters because the same coefficients that define the filter's poles go through quantisation. A coarsely quantised coefficient shifts a pole's location, which changes the frequency response, as Q8.8 demonstrates in Fig. 3 [1].

The within-fixed-point comparison makes the allocation cost concrete. Q3.13 and Q8.8 use identical memory (2000 bytes) but differ by orders of magnitude in MSE. Q8.8 allocates 8 bits to an integer range ($[-128, 128)$) that a normalised passband sine wave barely needs, leaving only 8 fractional bits where finer resolution is required. Q3.13 allocates 13 fractional bits, the safer match for this filter's coefficient range [3]. With the corrected half-precision simulation, float16 reaches 53.68 dB SNR at 2000 bytes, higher than Q3.13 (44.87 dB) because the exponent adapts precision sample-by-sample [4], yet still far below float32.

B. RQ2: Observed Quantisation Errors

The accuracy ranking is $\mathrm{float32} \gg \mathrm{float16} > \mathrm{Q3.13} \gg \mathrm{Q8.8}$ (Table I). Three observations stand out.

First, the pattern of error differs structurally between fixed and floating-point (Figs. 1–2). Fixed-point error is periodic with roughly uniform step size; floating-point error is irregular and signal-dependent because the quantisation step scales with magnitude [1]. The histograms make the spread of $e[n]$ explicit. Both are quantisation noise, but with different spectral properties relevant to audio.

Second, the frequency response (Fig. 3) shows that Q8.8 coefficient quantisation shifts poles enough to deviate visibly from the reference. Q3.13, float16, and float32 remain close to the reference, consistent with the idea that IIR coefficient wordlength strongly shapes filter accuracy [1].

Third, float32 at 136.6 dB SNR is effectively indistinguishable from the float64 baseline for this filter. It uses twice the memory of the 16-bit formats (4000 vs. 2000 bytes), but on any system with an FPU it remains the appropriate default.

VI. Conclusion

The fixed vs. floating-point decision is a genuine efficiency–accuracy trade-off. In our simulation, Q3.13 achieved 44.87 dB SNR at 2000 bytes using only integer arithmetic, demonstrating that a carefully chosen fixed-point format can be a viable option for embedded targets without an FPU when the binary-point position is matched to the signal range. float32 achieved 136.6 dB SNR in our experiment, effectively matching the float64 reference baseline, at twice the memory cost of the 16-bit formats (4000 vs. 2000 bytes). On any system with an FPU this trade-off is appropriate when precision is the priority. These conclusions hold for this specific 4th-order Butterworth filter and 50 Hz sine wave test signal. Results for higher-order designs, Chebyshev filters, or signals with wider dynamic range may differ and represent a direction for future work.

The more important finding remains *within* fixed-point: Q8.8 and Q3.13 cost the same in memory and word length, yet Q8.8 is unsuitable for this IIR application; its large SNR deficit stems from mismatched integer vs. fractional bit allocation. Format selection within fixed-point is primary, not secondary. Future work should validate efficiency claims on embedded hardware (e.g., Cortex-M0 without FPU vs. Cortex-M4 with FPU) and extend the study to higher-order or Chebyshev filters to further stress pole sensitivity [1].

IMPLEMENTATION

Code and notebooks:
https://github.com/SepehrF26/Group_3_DCA.git

REFERENCES

[1] S. W. Smith, *The Scientist and Engineer's Guide to Digital Signal Processing*, 1st ed. California Technical Publishing, 1997. ISBN 978-0-9660176-3-3. Ch. 28, "Fixed versus Floating Point"; companion material http://www.dspguide.com/ch28/4.htm (accessed Apr. 9, 2026).

[2] Analog Devices, "Fixed-point vs. floating-point digital signal processing," Technical Article, 2007. https://www.analog.com/en/resources/technical-articles/fixedpoint-vs-floatingpoint-dsp.html (accessed Apr. 9, 2026).

[3] G. Frantz and R. Simar, "Fixed-point vs. floating-point digital signal processing," Texas Instruments, Application Rep. SPRY061, Dec. 2007. https://www.ti.com/lit/wp/spry061/spry061.pdf (accessed Apr. 9, 2026).

[4] *IEEE Standard for Floating-Point Arithmetic*, IEEE Std 754-2019 (Revision of IEEE 754-2008). IEEE Standards Association, Jul. 2019. DOI: https://doi.org/10.1109/IEEESTD.2019.8766229.

[5] D. Goldberg, "What every computer scientist should know about floating-point arithmetic," *ACM Computing Surveys*, vol. 23, no. 1, pp. 5–48, Mar. 1991. DOI: https://doi.org/10.1145/103162.103163.

[6] A. V. Oppenheim and R. W. Schafer, *Discrete-Time Signal Processing*, 3rd ed. Upper Saddle River, NJ, USA: Pearson, 2010. ISBN 978-0-13-198842-2.

[7] R. W. Penney, *SPFPM: Simple Python Fixed-Point Module* (software), GitHub repository. https://github.com/rwpenney/spfpm (accessed Apr. 9, 2026). Pin the release tag or commit SHA in your report if your module requires reproducibility beyond "main".

[8] P. Virtanen *et al.*, "SciPy 1.0: Fundamental algorithms for scientific computing in Python," *Nature Methods*, vol. 17, no. 3, pp. 261–272, Mar. 2020. DOI: https://doi.org/10.1038/s41592-019-0686-2.

[9] C. R. Harris *et al.*, "Array programming with NumPy," *Nature*, vol. 585, pp. 357–362, Sep. 2020. DOI: https://doi.org/10.1038/s41586-020-2649-2.